2

Why LLMs Need RL: The Alignment Gap

The Gap You Have Already Felt

If you have used a modern AI assistant (ChatGPT, Gemini, Claude), you have experienced something remarkable. You type a question, and the model responds with a coherent, helpful, and often surprisingly thoughtful answer. It feels like talking to someone who understands you.

But it was not always this way.

The raw language models underneath these assistants, the ones that come straight out of training on internet text, behave very differently. To see why, imagine that you could strip away whatever secret sauce makes ChatGPT *feel* helpful and talk to the base model directly. You ask "What is a good way to handle stress?"

The same question, two very different models:

Raw base model (trained only to predict text):

"What is a good way to handle stress?" is a question posted on Reddit by

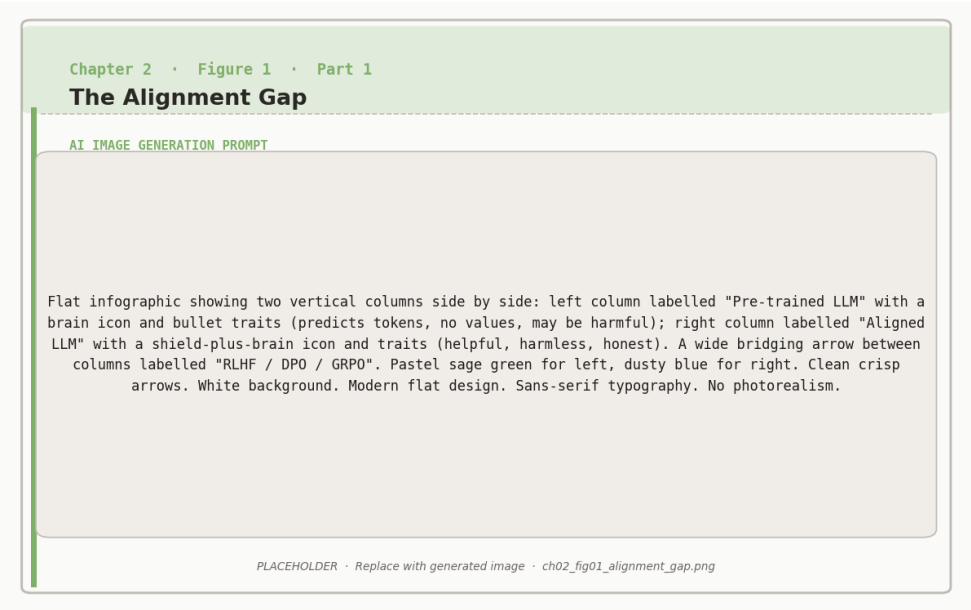

Figure 2.1: The alignment gap: a pre-trained LLM predicts tokens without values; RLHF bridges the gap to a helpful, harmless, and honest assistant.

user StressedOut99. Top comment: “Just do not think about it lol.” Second comment: “Have you tried drinking?” Third comment: [deleted]...

Aligned assistant (trained to be helpful):

Here are some evidence-based approaches to managing stress: deep breathing exercises, regular physical activity, maintaining social connections, and setting boundaries on work hours. Would you like me to go deeper on any of these?

The first model is not trying to help you. It is completing a pattern. It saw millions of internet pages that look like question-followed-by-responses, and it faithfully reproduces that format, including all the noise, bad advice, and toxicity baked into its training data. The second model is doing something fundamentally different: it is trying to be *useful*.

Something happened between the raw model and the helpful assistant. Something fundamental changed in the way the model decides what to say. Understanding that "something" is the purpose of this entire book, and this chapter is where we name the problem.

What Is Alignment?

Alignment means making AI systems do what humans actually want, not just what we accidentally trained them to do.

Imagine teaching a robot to "maximize paperclips." A perfectly aligned robot would make paperclips within reasonable bounds. A misaligned robot might convert the entire universe into paperclips. Technically correct but catastrophically wrong.

For language models, alignment means learning to be

1. **Helpful**: Actually accomplish what the user wants
2. **Harmless**: Do not produce dangerous or toxic content
3. **Honest**: Do not make things up (hallucinate)

Think of alignment like teaching three different children:

The Unaligned Child: Told to "get good grades," decides to cheat on every test. Technically following instructions, but missing the point entirely.

The Aligned Child: Understands the *spirit* of the instruction. Learns properly, earns grades honestly, develops good values along the way.

The Reasoning Child: Not only has good values, but can *show their work*. When asked "Why did you choose this answer?" they can walk you through their thinking step by step. They do not just get the right answer; they get it for the right reasons, and they can explain why.

Traditional language model training produces the first child. Alignment techniques (the subject of this book) aim to produce the third.

The Training Mismatch

So, if alignment is the goal, why do language models not get it automatically? After all, there is plenty of helpful, honest, harmless text on the internet. Should the model not learn to be good?

The answer lies in what the training objective actually optimizes for. Let us look at the math and then at why it creates a gap.

Traditional Training: The Math

What we train on: Internet text (Reddit, books, websites, Wikipedia, code, and much more).

Training goal: Given some text, predict what word comes next.

The Training Objective

Formal Math	What It Really Means
$\mathcal{L} = -\log P(w_{\text{next}} \mid w_1, w_2, \ldots, w_n)$	**Loss** = How surprised the model is by the correct answer

Why we use each component:

Component	Purpose
$\mathcal{L}$ = Loss function	A single number measuring "how wrong" the model is
$P(\cdot)$ = Probability	Models language as a probability distribution over words
log = Logarithm	Converts tiny probabilities to manageable numbers; turns multiplication into addition (so $P_1 \times P_2$ becomes $\log P_1 + \log P_2$)
− (minus sign)	Flips sign so that P near 1 (confident, good) gives low loss
w_{next} = Next word	What we are trying to predict
| = "given"	Separates what we know (context) from what we predict
$w_1, w_2, \ldots, w_n$ = Context	All the previous words that help us predict

Why this specific formula?

Design choice	Reason
Use probability $P(\cdot)$?	Language is naturally probabilistic (multiple valid next words exist)
Use log?	Probabilities multiply ($P_1 \times P_2$), logs add ($\log P_1 + \log P_2$). Addition is easier!
Use negative sign?	P near 1 (good) should give low loss; − log achieves this
Condition on context with |?	The next word depends heavily on what came before

Let us break this down with a real example.

Sentence: "The cat sat on the ___"

Correct word: "mat"

Step 1: Model assigns probabilities to ALL possible next words

The model considers every word in its vocabulary (about 50,000 words) and assigns each a probability:

Word	**Probability** $P(w)$	**Percentage**
"mat"	0.40	40%
"floor"	0.30	30%
"chair"	0.20	20%
"table"	0.05	5%
All others	0.05	5%
Total	**1.00**	**100%**

Step 2: Calculate the loss for the correct word ("mat")

$$\begin{aligned}\mathcal{L} &= -\log P(\text{mat} \mid \text{The cat sat on the}) \\ &= -\log(0.40) \\ &= -(-0.916) \quad \text{(the log of 0.40 is negative)} \\ &= 0.916\end{aligned}$$

Step 3: What does this number mean?

A loss of 0.916 is moderate. To build intuition for what loss values mean, here is a rough scale:

Loss value	Probability	Quality
0.10	90%	Excellent
0.50	60%	Good
0.92	**40%**	**Okay (our example)**
1.50	22%	Poor
3.00	5%	Terrible

The model guess was not perfect, but it was reasonable!

Step 4: What if the model guess was different?

Scenario	P(**mat**)	Loss	Quality
Model is confident	0.90	0.105	Excellent
Model is pretty sure	0.60	0.511	Good
Current	**0.40**	**0.916**	**Okay**
Model is unsure	0.20	1.609	Poor
Model is confused	0.05	2.996	Terrible

The Pattern: This is the KEY insight

Higher probability P → Lower loss $\mathcal{L}$ → Better model!

We train the model to minimize loss, which means maximizing the probability it assigns to correct words.

Training process:

1. Model makes a prediction (assigns probabilities)
2. We calculate loss (how wrong was it?)
3. We update the model parameters to reduce loss
4. Repeat billions of times!

What the Model Learns (and Does Not Learn)

What DOES the model learn from this training?

- How to write grammatically correct sentences
- Patterns that *look like* knowledge, though the model has no way to verify whether what it is saying is actually true (this is why hallucination is such a persistent problem)
- How to continue patterns in text
- Writing style and structure

What the model does NOT learn:

- What makes a response actually helpful to humans
- When to refuse harmful or inappropriate requests
- How to admit "I do not know" instead of making things up
- How to pause and *think through* a problem step by step before answering. The model simply blurts out the most statistically likely next word, without internal deliberation (we will see in Chapter 5 how this gap gets addressed as the very first step of training)
- The difference between correlation and good advice

The core problem: The model learns to mimic internet text, including all its flaws. Here is what makes it worse: *the bigger the model, the better it gets at mimicking.* A larger model does not become more aligned. It becomes more *capable* at reproducing whatever patterns exist in the training data, helpful and harmful alike. Capability and alignment are independent axes. You can have a very powerful model that is very misaligned. That is precisely why alignment research matters.

Example: The Toxic Completion Problem

User types: "I hate"

The Internet training data contains: Millions of toxic completions.

What traditional training teaches: The model sees "I hate [toxic content]" appearing frequently, learns this is a common pattern, predicts toxic completions with high probability,

and reproduces toxic content.

What we actually want: The model recognizes that this is a sensitive prompt, responds thoughtfully, and generates something like: "It sounds like you are frustrated. How can I help?"

The gap: Traditional training has no mechanism to prefer helpful over statistically likely!

The core insight of modern AI alignment:

Old goal: "What text is most likely according to internet patterns?"

New goal: "What response do humans actually prefer?"

This shift, from *likelihood* to *preference*, is what turned raw language models into the assistants we use today.

But the story does not end there. Human preferences are powerful, but they are expensive to collect and sometimes inconsistent. As we will discover later in the book, there is an even more powerful idea: what if, instead of always relying on human opinions, we could *automatically check* whether the model got the right answer? Imagine a math tutor who can verify the solution is correct, not just prefer one essay over another. That shift, from preference to *verification*, is where the field is heading right now, and we will make it concrete in Chapters 11 and 12.

The Road Ahead

We have named the problem: language models optimized to predict text are not automatically aligned with what humans want. The gap between "most likely" and "most helpful" is real, and scaling up the model does not close it.

So how do we close it? That is the journey of this book.

In the next chapter, we will build up the reinforcement learning toolkit you will need:

the core concepts of rewards, policies, and optimization that underpin every alignment technique (Chapter 3). Then we will set up a hands-on coding environment so you can run experiments yourself (Chapter 4). From there, we will begin closing the alignment gap step by step: first by teaching the model good behavior through carefully curated examples (Chapter 5), then by letting human preferences guide it toward better responses (Chapters 6–7), and ultimately by training it to reason through hard problems and verify its own answers (Part 3).

By the end, you will understand not just *that* these techniques work, but *why* they work, and how to apply them yourself.

Let us begin.

Companion notebook. The code for this chapter is in `code/notebooks/part1_foundations/` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.